Adaptive Traffic Blockchain

System Implementation Guide

December 20, 2025

Groupe 5

1 System Architecture

Network Structure

```
Orderer (Solo Consensus)
    |
    +-- Org1 (Traffic Authority)
    |   - peer0.org1.example.com:7051
    |
    +-- Org2 (Emergency Services)
        - peer0.org2.example.com:9051

Channel: traffic-channel
Chaincode: traffic-light v1.1
```

2 Implementation Steps

2.1 cog Project Setup

Step 1: Navigate to Project Directory

```
cd ~/adaptive-traffic-blockchain
```

Step 2: Install Simulator Dependencies

```
cd simulator
npm install
cd ..
```

Step 3: Install Chaincode Dependencies

```
cd chaincode/traffic-light
npm install
cd ../..
```

2.2 network-wired Network Configuration

Step 4: Generate Network Artifacts

```
cd network
./scripts/generate-artifacts.sh
```

Step 5: Start Fabric Network

```
./scripts/network-up.sh
```

Step 6: Create Channel

```
./scripts/create-channel.sh
```

Step 7: Deploy Chaincode

```
./scripts/deploy-chaincode.sh
```

Step 8: Initialize Ledger

```
./scripts/test-chaincode.sh init
```

2.3 user-shield Identity Management

Step 9: Import Admin Identities

```
cd ../simulator
node importAdminIdentities.js
```

Step 10: Enroll Admins

```
node enrollBothAdmins.js
```

Step 11: Register Users

```
node registerBothUsers.js
```

2.4 rocket Application Deployment

Step 12: Start Traffic Simulator (Port 3000)

```
npm start &
```

Step 13: Start Traffic Dashboard (Port 8080)

```
cd ../visualization
python3 -m http.server 8080 &
```

2.5 check-circle Testing and Verification

Step 14: Test Hash Functions

```
cd ../chaincode/traffic-light
python3 quick_test.py
```

Step 15: Run Full Test Suite

```
cd ../../network
./scripts/test-chaincode.sh test
```

3 Access Points

Application URLs

- **Dashboard:**
  `http://localhost:8080`
- **Hash Demo:**
  `http://localhost:8080/hash-demo.html`
- **API Endpoint:**
  `http://localhost:3000`